

Наблюдаемость Go-сервисов с OpenTelemetry

Разработка микросервисов на Go



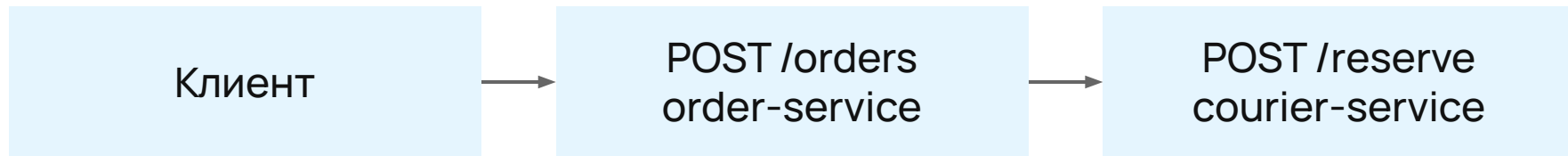
Цели занятия:

- Узнать как строится мониторинг приложения
- Получить практические знания по тому, как использовать мониторинг для определения и реагирования на проблемы с системой



Пользовательская проблема

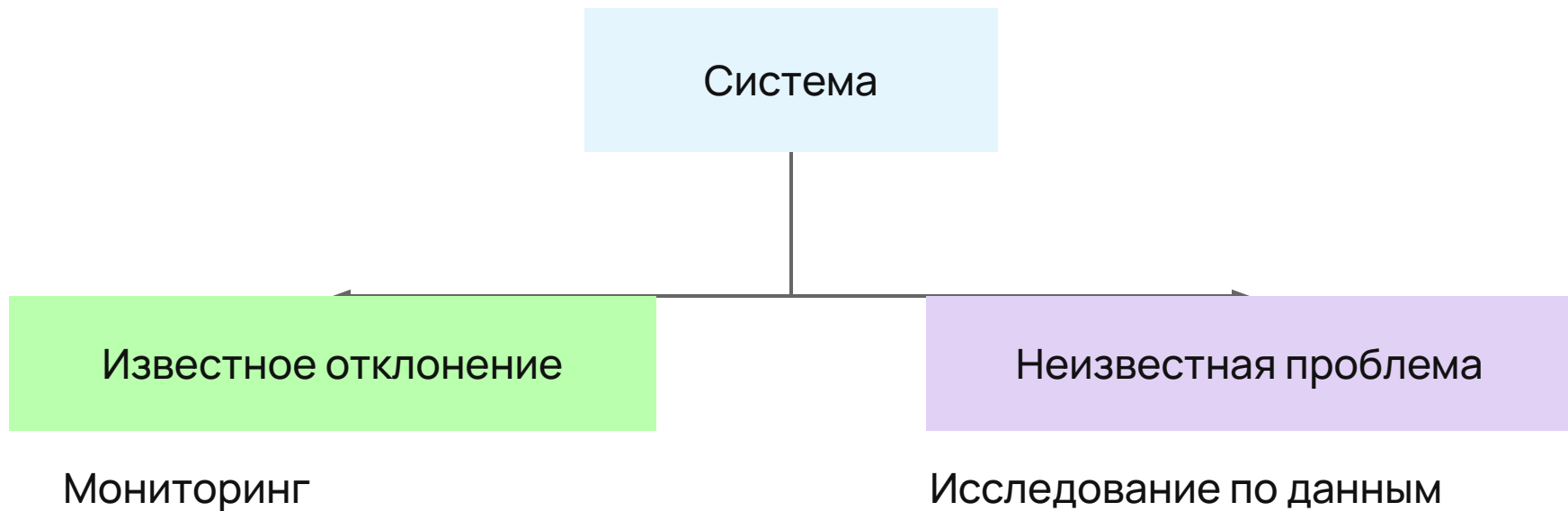
2 секунды



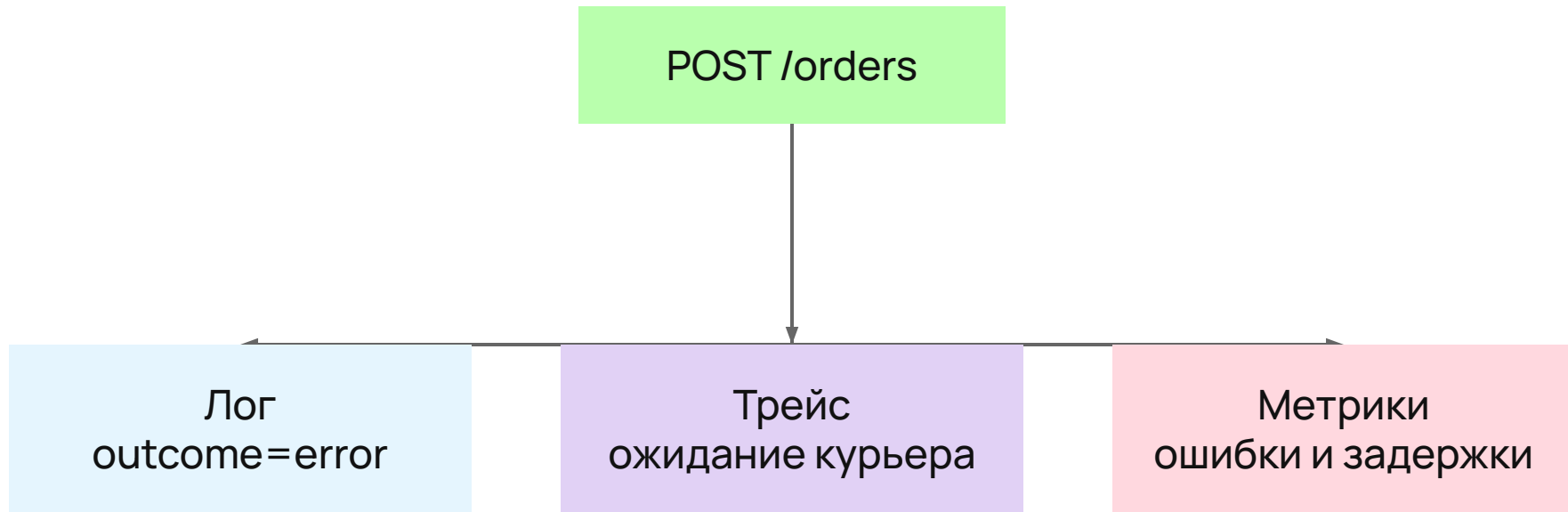
503

Процессы запущены. Заказ завершился ошибкой.

Мониторинг и наблюдаемость



Один заказ, три сигнала



Структурированный лог

```
{  
  "time": "2026-10-03T21:25:07+03:00",  
  "level": "INFO",  
  "service.name": "order-service",  
  "body": "order completed",  
  "outcome": "error"  
}
```

Loki: поиск логов одного сервиса

```
{service_name="courier-service"}
```

Время

Level

Trace ID

2026-10-03 21:30:22.418	INFO	9a4ba297e4a81690467c624b4946d3f6
2026-10-03 21:30:22.333	INFO	243a1e95e554041538e2da69f10bbbd1
2026-10-03 21:30:22.055	INFO	be870735d58c673aa83283c912cdaf4a

Как связать записи разных сервисов с одним запросом?

Трейс и спаны

POST /orders



orders.create



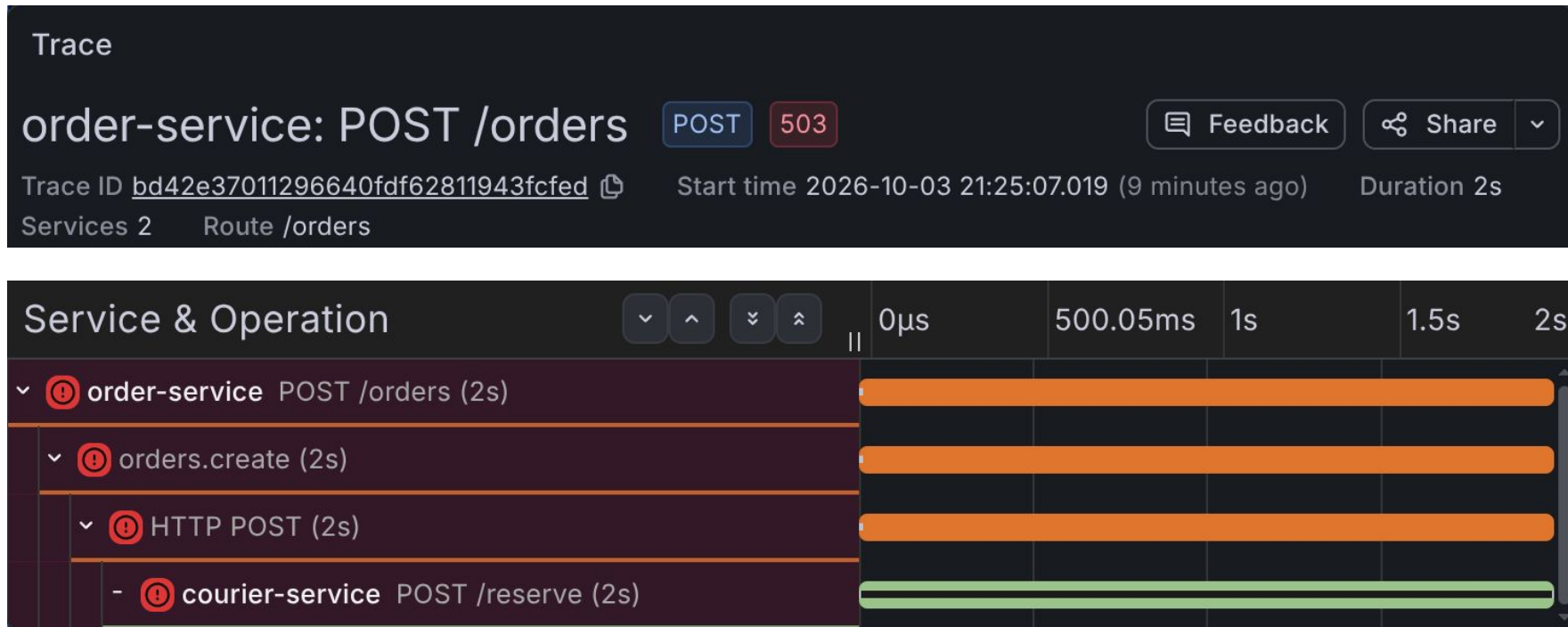
HTTP POST



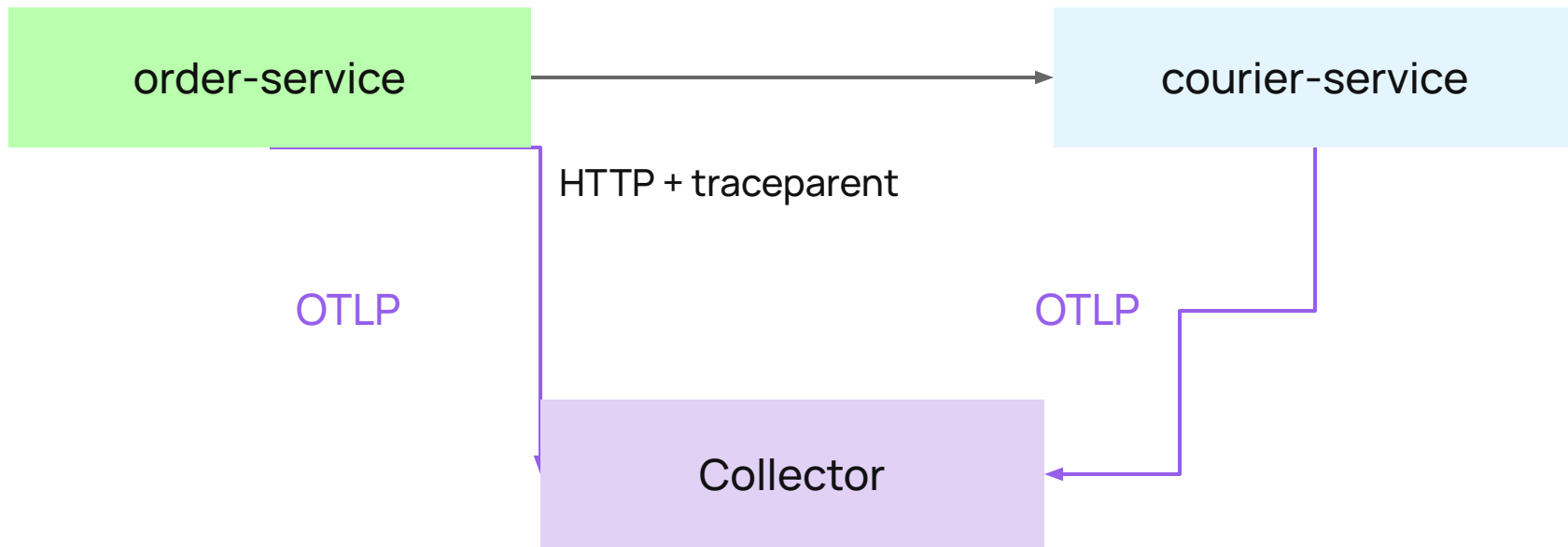
POST /reserve



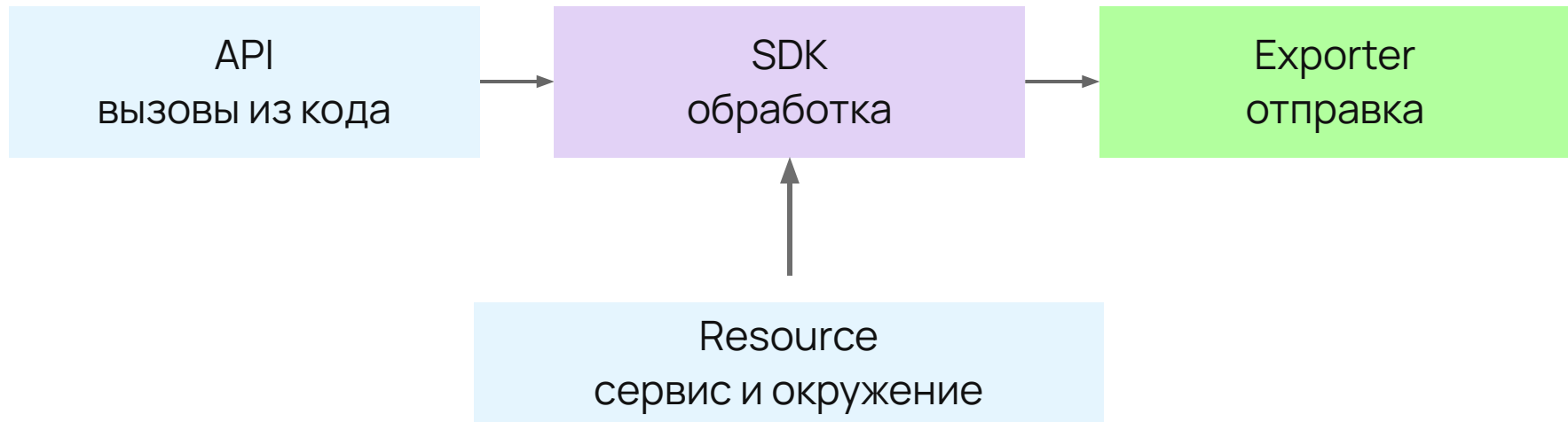
Jaeger: один запрос, четыре спана



Контекст запроса и экспорт



OpenTelemetry: от кода к экспорту



Код вызывает API · SDK обрабатывает · exporter отправляет

Семантические соглашения

Стандартные атрибуты: semconv

```
res := resource.NewWithAttributes(  
    semconv.SchemaURL,  
    semconv.ServiceName("order-service"),  
    semconv.ServiceNamespace("tripgo"),  
)
```

Scope и атрибуты операции

```
scope := "tripgo/orders"  
attribute.String("outcome", outcome)
```

Трейсы: SDK при запуске

```
traceExporter, err := otlptracehttp.New(ctx)
if err != nil { return nil, shutdown, err }
```

```
tp := sdktrace.NewTracerProvider(
    sdktrace.WithResource(res),
    sdktrace.WithBatcher(traceExporter),
)
otel.SetTracerProvider(tp)
```

HTTP-инструментирование

```
otel.SetTextMapPropagator(propagation.TraceContext{})
```

```
client.Transport = otelhttp.NewTransport(client.Transport)
```

```
mux.Handle("POST /orders",  
    otelhttp.NewHandler(create, "POST /orders"),  
)
```

Серверный спан + клиентский спан + propagation

Бизнес-операция: ручной спан

```
ctx, span := tracer.Start(r.Context(), "orders.create")  
defer span.End()  
  
err := reserveCourier(ctx, courierURL, client)
```

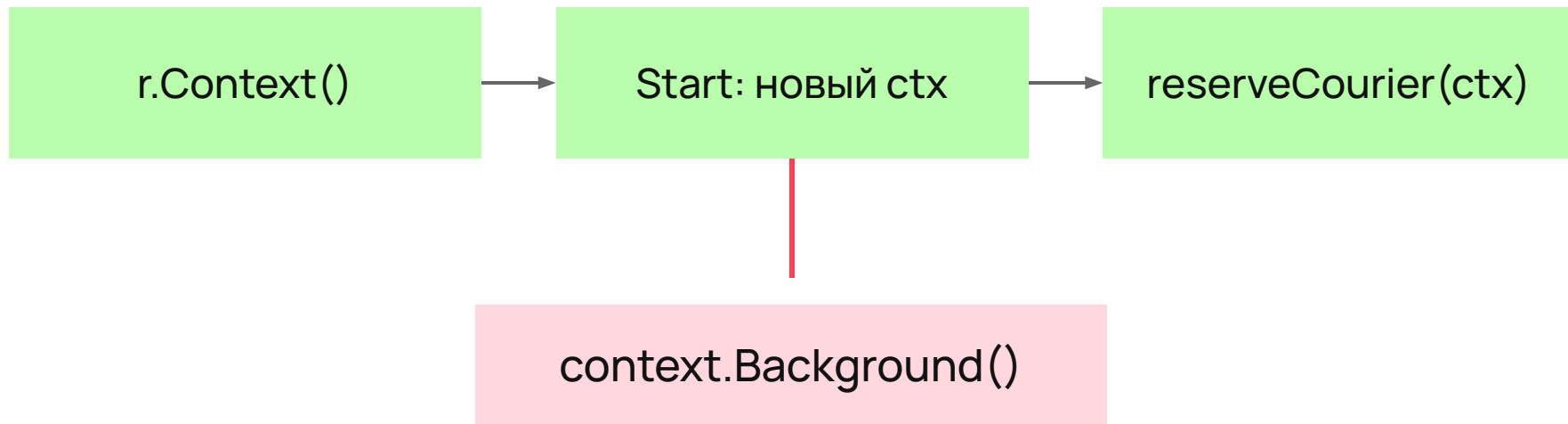
Новый ctx передаём дальше

Ошибка: событие и статус

```
outcome := "success"
if err != nil {
    outcome = "error"
    span.RecordError(err)
    span.SetStatus(codes.Error, "courier reservation failed")
}

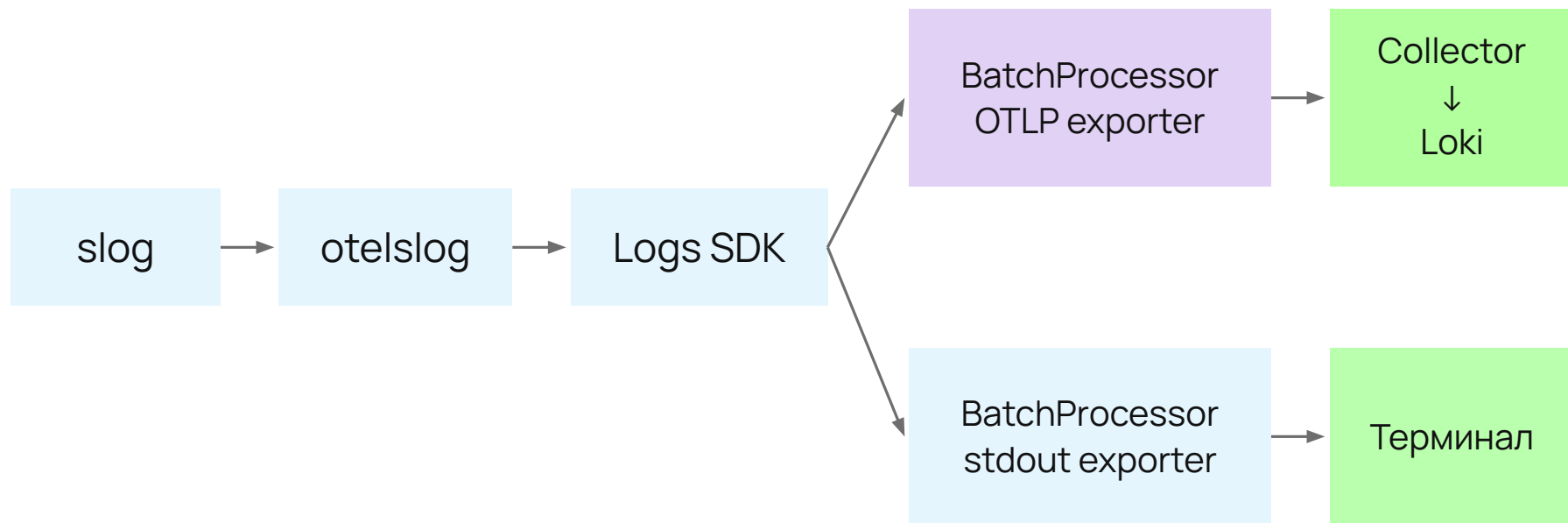
result := attribute.String("outcome", outcome)
span.SetAttributes(result)
```

Контекст связывает операции



Потеря текущего контекста разрывает родительство и отмену

Логи: slog и OpenTelemetry



OTLP — доставка в Loki · stdout — локальная диагностика

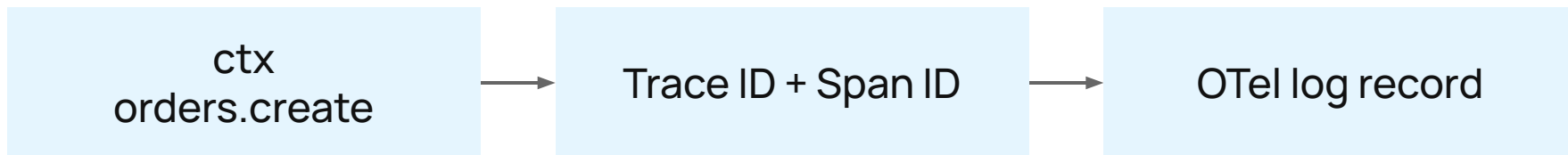
Логи: подключение моста

```
lp := sdklog.NewLoggerProvider(  
    sdklog.WithResource(res),  
    sdklog.WithProcessor(  
        sdklog.NewBatchProcessor(otlpExporter)),  
    sdklog.WithProcessor(  
        sdklog.NewBatchProcessor(stdoutExporter)),  
)  
logger := otelslog.NewLogger(scope,  
    otelslog.WithLoggerProvider(lp))
```

Фрагмент: OTLP- и stdout-экспортёры уже созданы

Лог внутри текущего спана

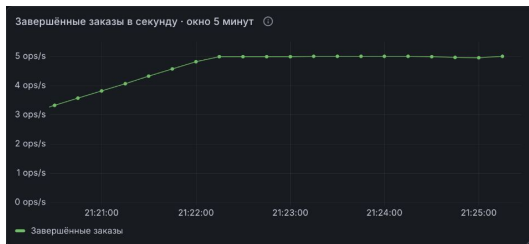
```
logger.InfoContext(ctx, "order completed", "outcome", outcome)
```



Loki → View trace → Jaeger

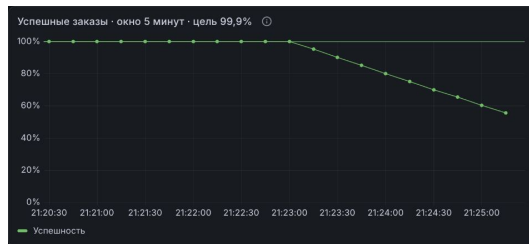
RED: общая картина

Rate



попытки / с

Errors



100% – успешность

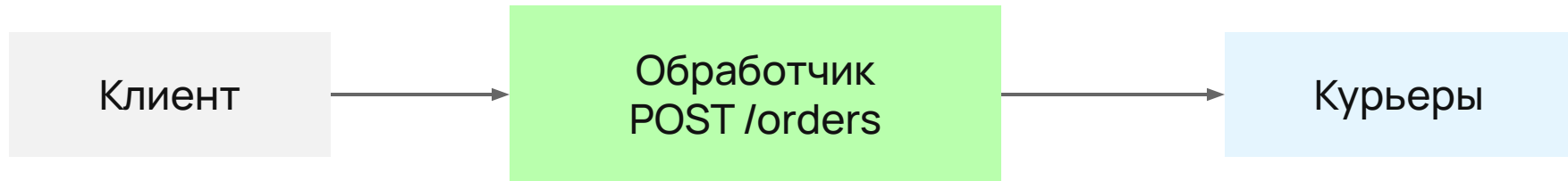
Duration



секунды

Граница измерения

Начало

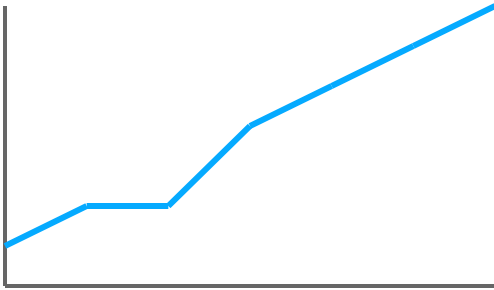


Результат: 201 / 503

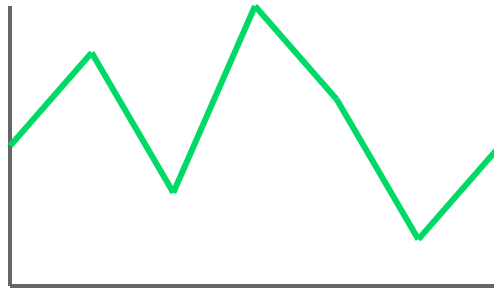
Учитываем завершённые попытки, дошедшие до обработчика

Counter, Gauge, Histogram

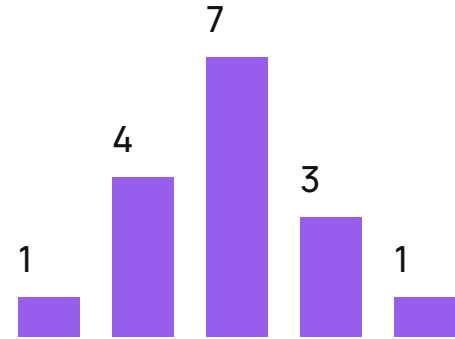
Counter



Gauge



Histogram



прибавления

текущее значение

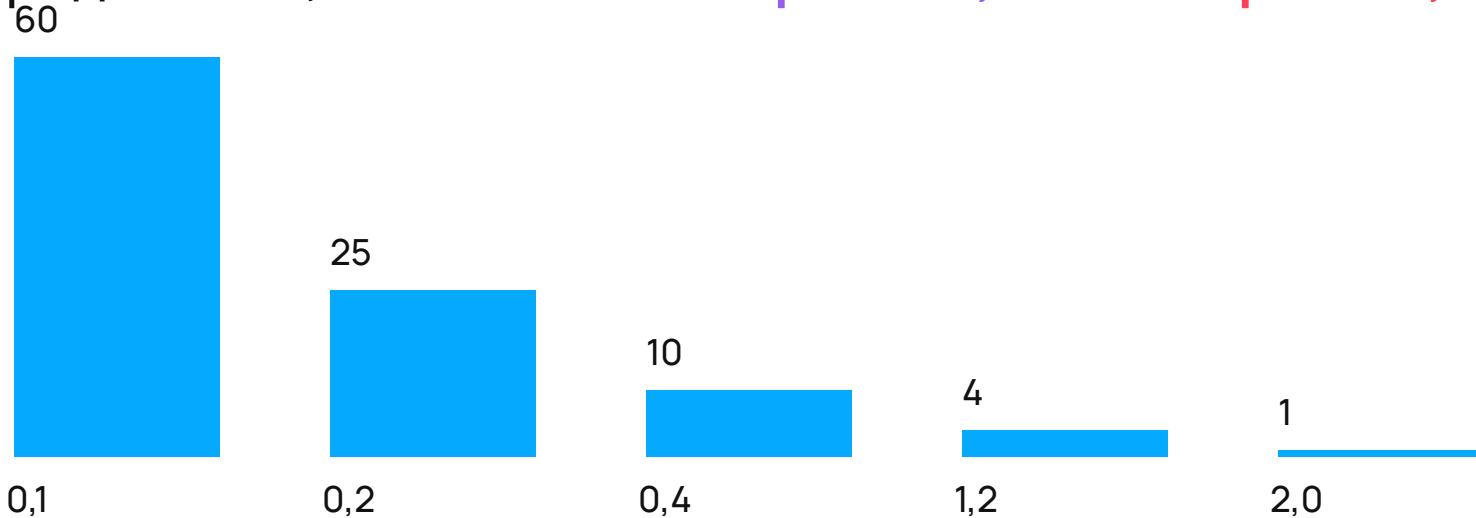
распределение

Среднее и хвост задержек

среднее = 0,218 с

p95 = 0,4 с

p99 = 1,2 с



100 условных запросов. Длительность, с

Метрики: SDK и экспорт

```
metricExporter, err := otelmetrichttp.New(ctx)
if err != nil { return nil, shutdown, err }

mp := sdkmetric.NewMeterProvider(
    sdkmetric.WithResource(res),
    sdkmetric.WithReader(
        sdkmetric.NewPeriodicReader(metricExporter)),
)
otel.SetMeterProvider(mp)
```

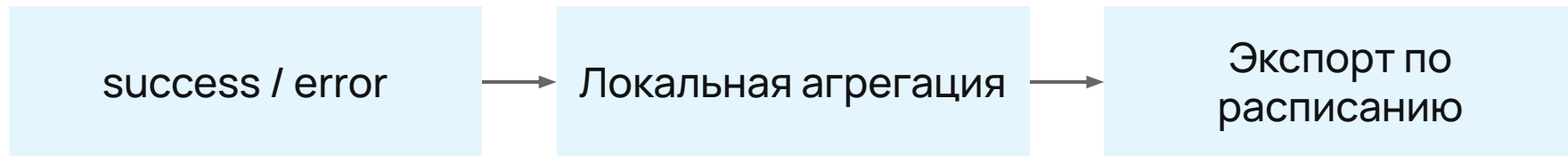
Counter и Histogram

```
meter := otel.Meter(scope)
attempts, err := meter.Int64Counter("orders.attempts")
if err != nil { return nil, err }

duration, err := meter.Float64Histogram("orders.duration",
    metric.WithUnit("s"),
    metric.WithExplicitBucketBoundaries(
        0.05, 0.1, 0.3, 0.8, 1, 2, 5),
)
if err != nil { return nil, err }
```

Одна завершённая попытка

```
attrs := metric.WithAttributes(  
    attribute.String("outcome", outcome))  
attempts.Add(ctx, 1, attrs)  
duration.Record(ctx, time.Since(started).Seconds(), attrs)
```



Кардинальность

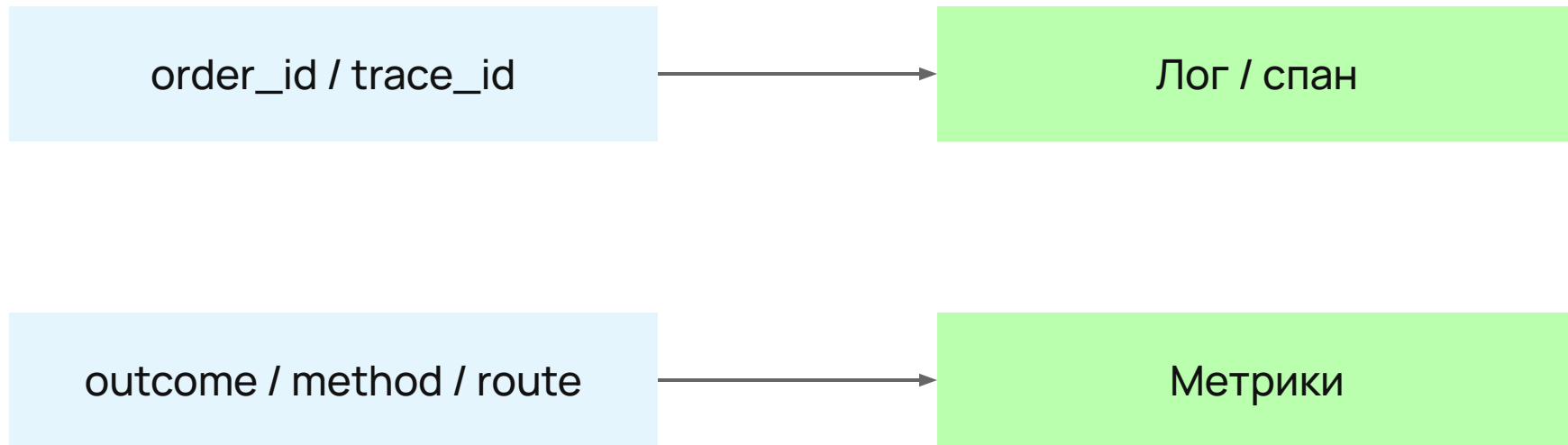
2 результата × 4 экземпляра = 8 рядов

outcome = success | error

order_id = ...

Каждый уникальный набор меток создаёт отдельный ряд

Поля событий и метрик



Пароли, токены и полные тела запросов исключаем

Накопительные бакеты

```
duration.Record(ctx, 0.42, attrs)
```

0

$\leq 0,1$

`_count +1`

0

$\leq 0,3$

`_sum +0,42 c`

+1

$\leq 0,8$

+1

≤ 1

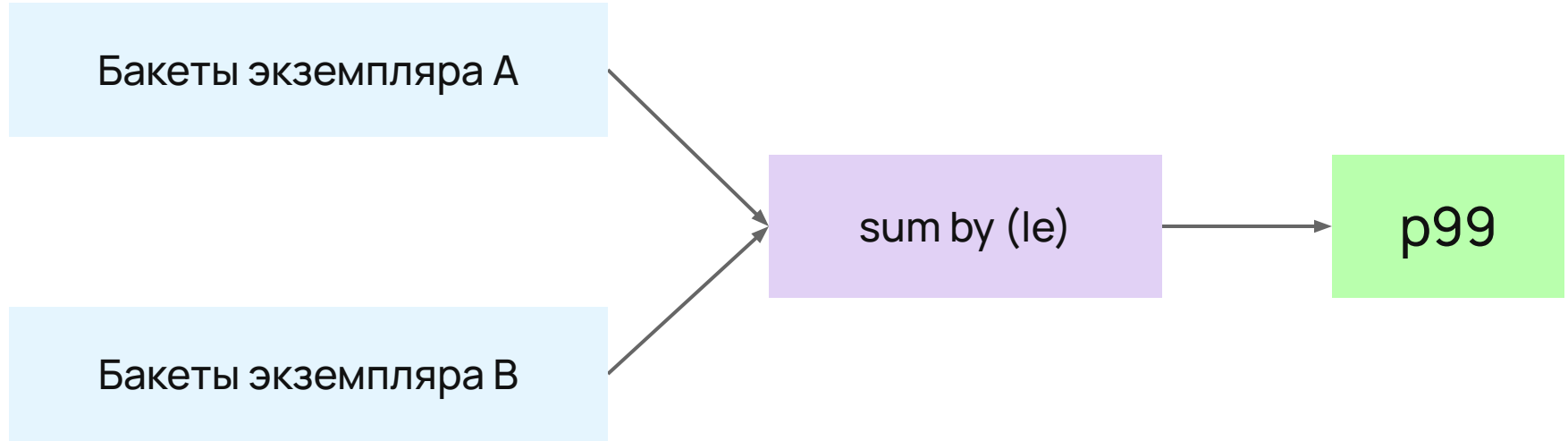
+1

≤ 2

+1

$+\text{Inf}$

Общий p99



Сначала объединяем распределения, затем вычисляем квантиль

PromQL: поток попыток

```
sum(rate/orders_attempts_total{  
  job="otel-collector",  
  exported_job="tripgo/order-service"  
}[5m]))
```

2 = две попытки в секунду

[5m] — окно на каждом шаге графика

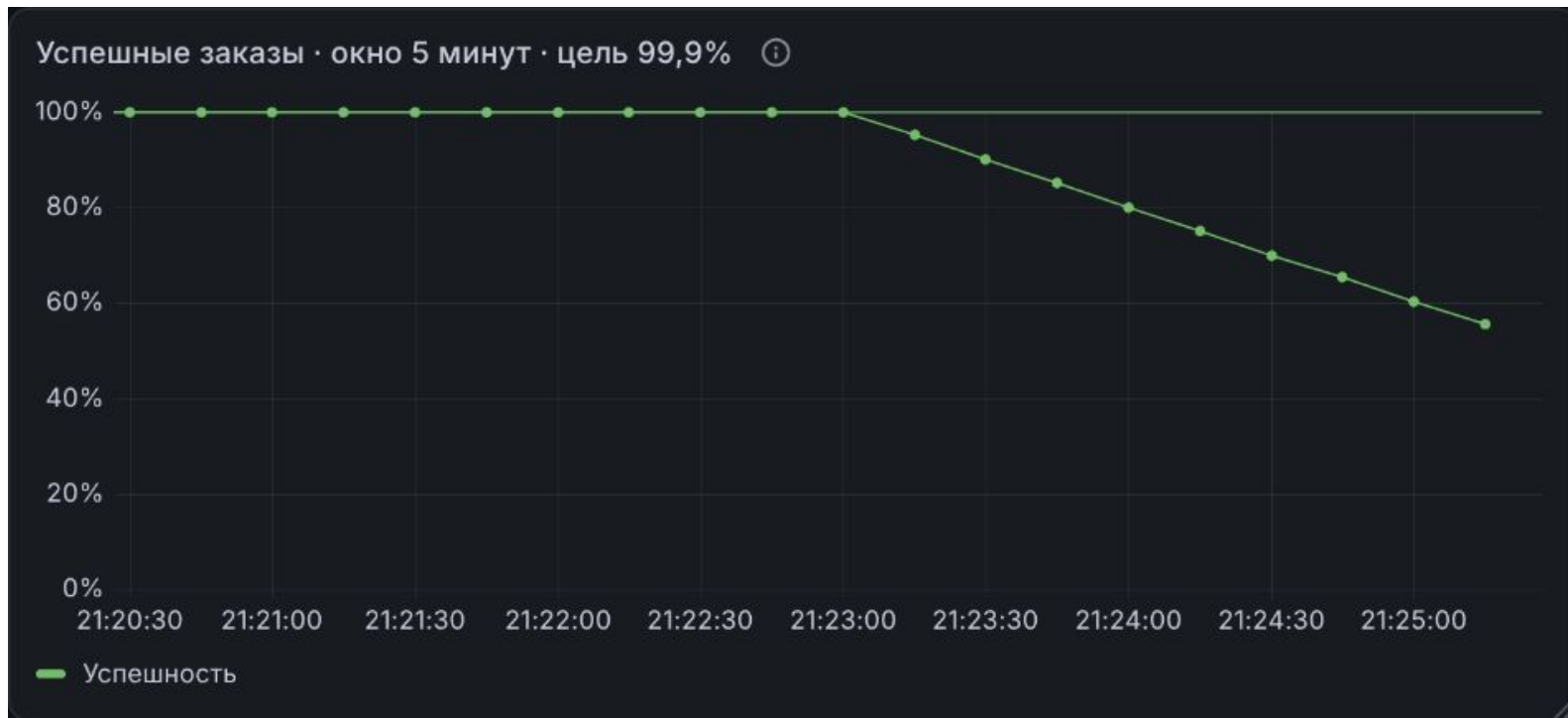
Grafana: поток попыток



PromQL: успешность

```
100 * sum(rate(orders_attempts_total{
    job="otel-collector", exported_job="tripgo/order-service",
    outcome="success"
}[5m]))
/
(sum(rate(orders_attempts_total{
    job="otel-collector", exported_job="tripgo/order-service"
}[5m])) > 0)
```

Grafana: успешность



PromQL: p99

```
histogram_quantile(  
    0.99,  
    sum by (le) (  
        rate(orders_duration_seconds_bucket{  
            job="otel-collector",  
            exported_job="tripgo/order-service"  
        }[5m])  
    )  
)
```

Grafana: хвост задержек



PromQL: успешные ≤ 800 мс

```
sum(rate(orders_duration_seconds_bucket{  
  job="otel-collector", exported_job="tripgo/order-service",  
  outcome="success", le="0.8"  
}[5m]))
```

```
sum(rate(orders_duration_seconds_count{  
  job="otel-collector", exported_job="tripgo/order-service"  
}[5m]))
```

Доля $\times 100 \rightarrow$ проценты

Если успешного ряда ещё нет

```
100 * (success_fast or (0 * all_attempts))  
/ (all_attempts > 0)
```

success_fast — скорость успешных ≤ 800 мс

all_attempts — скорость всех попыток

Есть успешные
рассчитываем %

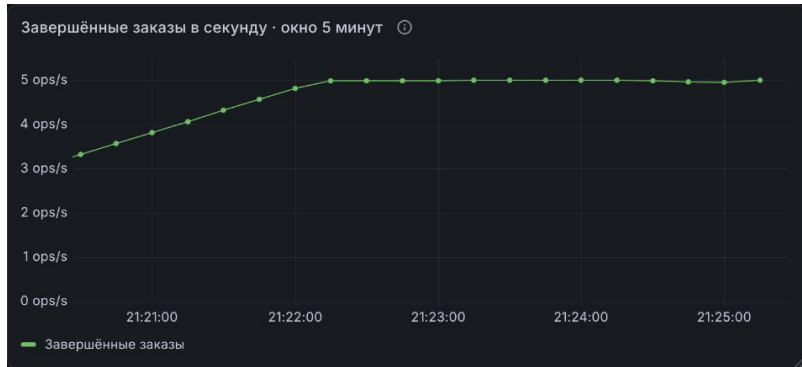
Только ошибки
0%

Нет попыток
нет значения

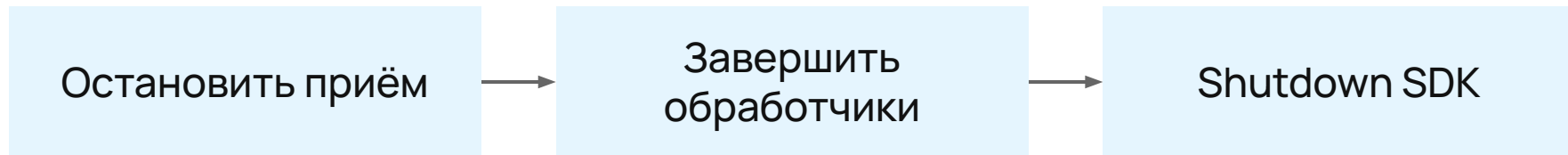
Grafana: быстрые успешные попытки



Четыре графика

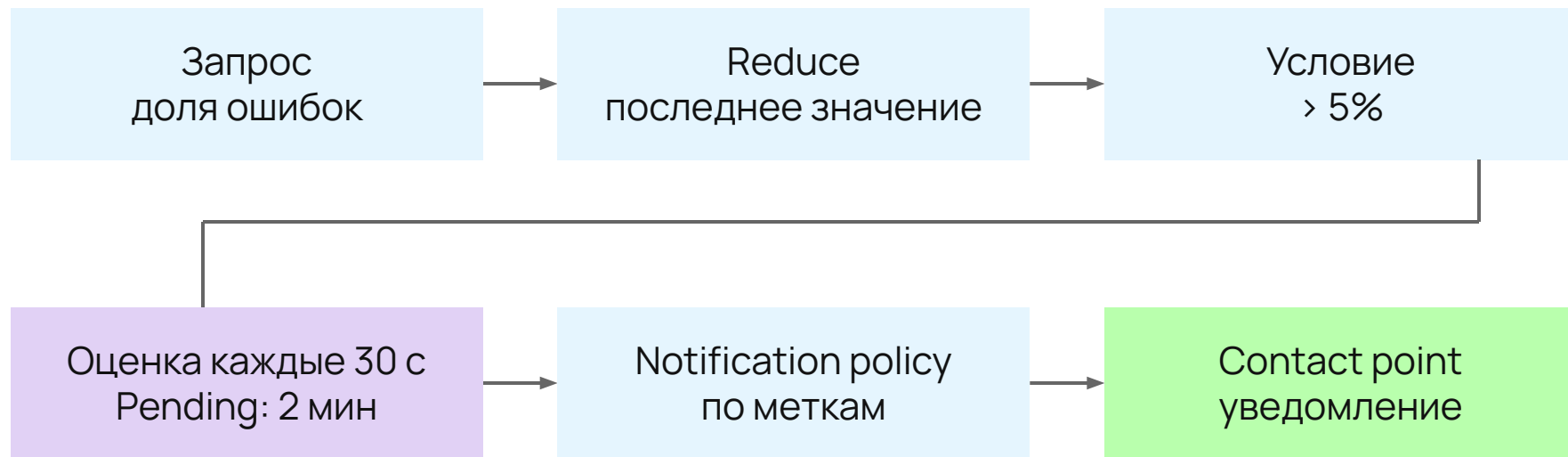


Завершение приложения



```
ctx, cancel := context.WithTimeout(context.Background(), timeout)
defer cancel()
```

Grafana Alerting: от запроса к уведомлению

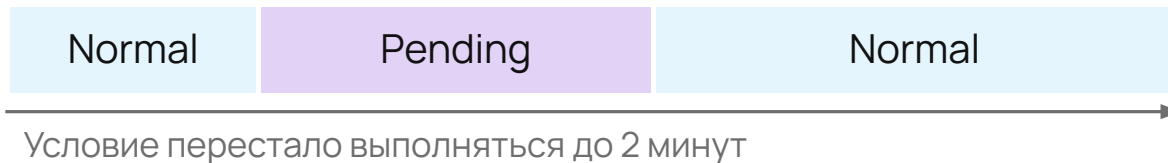


Учебный пример порогов

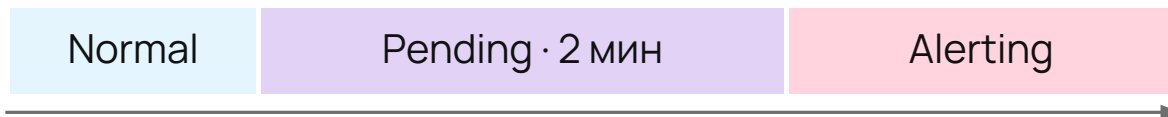
Короткий всплеск и устойчивое нарушение

Короткий всплеск

Условие > 5% проверяется каждые 30 секунд



Устойчивое нарушение



No Data / Error — отдельная политика обработки

Уведомление, с которым можно действовать

FIRING · order-service

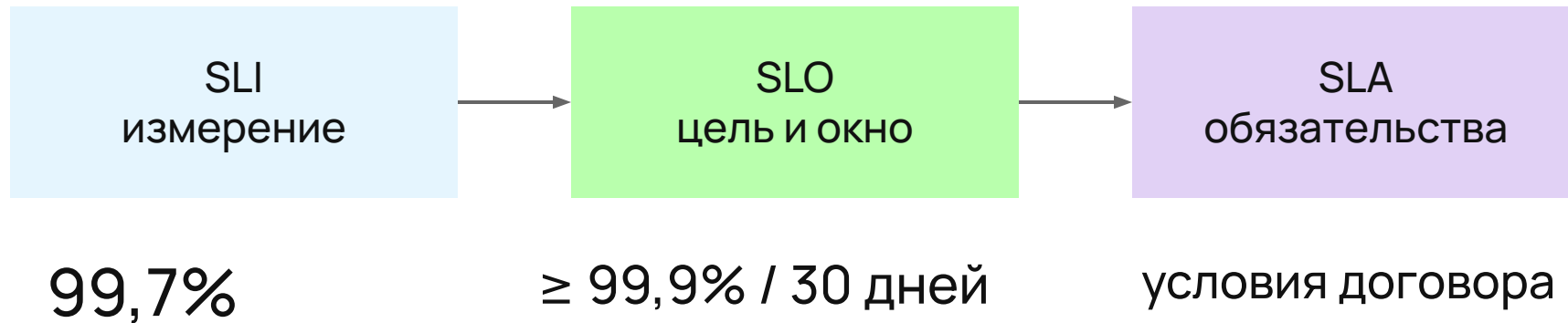
Ошибки оформления заказа: 8,2%

Порог > 5% · нарушение держится 2 минуты

Dashboard → Runbook

Пример уведомления · условные значения

SLI, SLO, SLA



Два SLI и две цели

успешные попытки

все допустимые попытки

$\geq 99,9\%$

успешные попытки ≤ 800 мс

все допустимые попытки

$\geq 99\%$

Окно: последние 30 дней

Бюджет ошибок

100 000 попыток $\times$ 0,1% = 100 ошибок

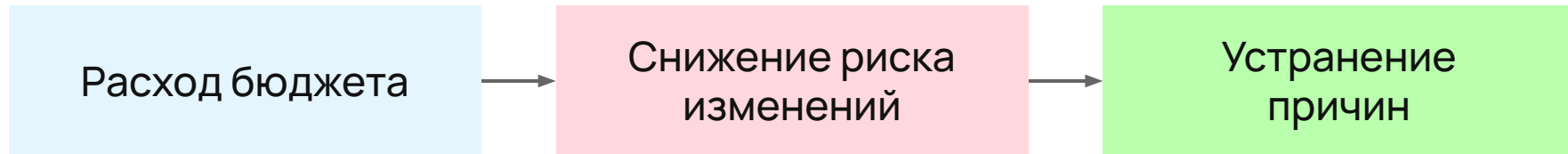
60 использовано

40 осталось

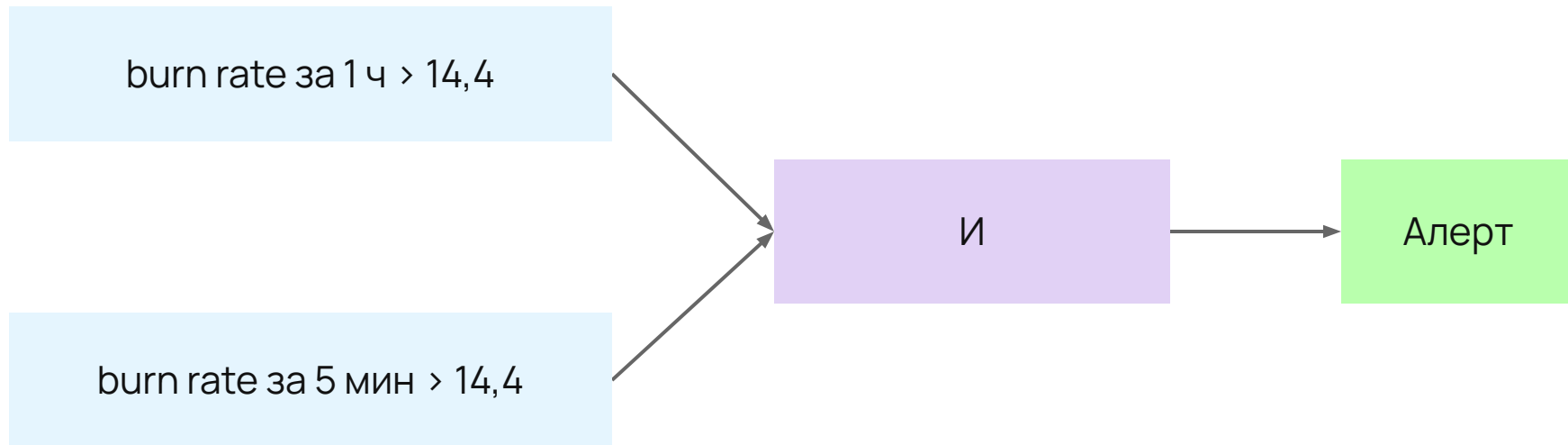
SLO 99,9%. Скользящее окно 30 дней.

Burn rate

$$1\% / 0,1\% = 10$$



Алерт на расход бюджета



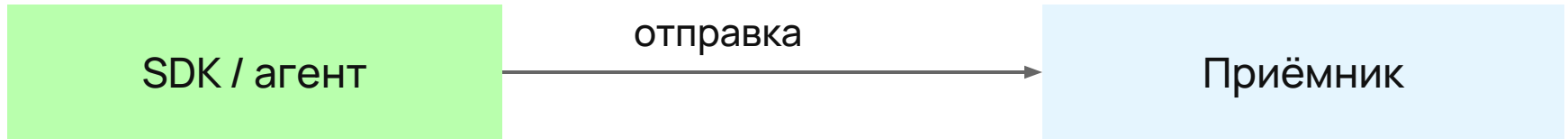
SLO 99,9% · окно 30 дней · учебный пример

OpenTelemetry: компоненты



Push и pull

Push: OTLP

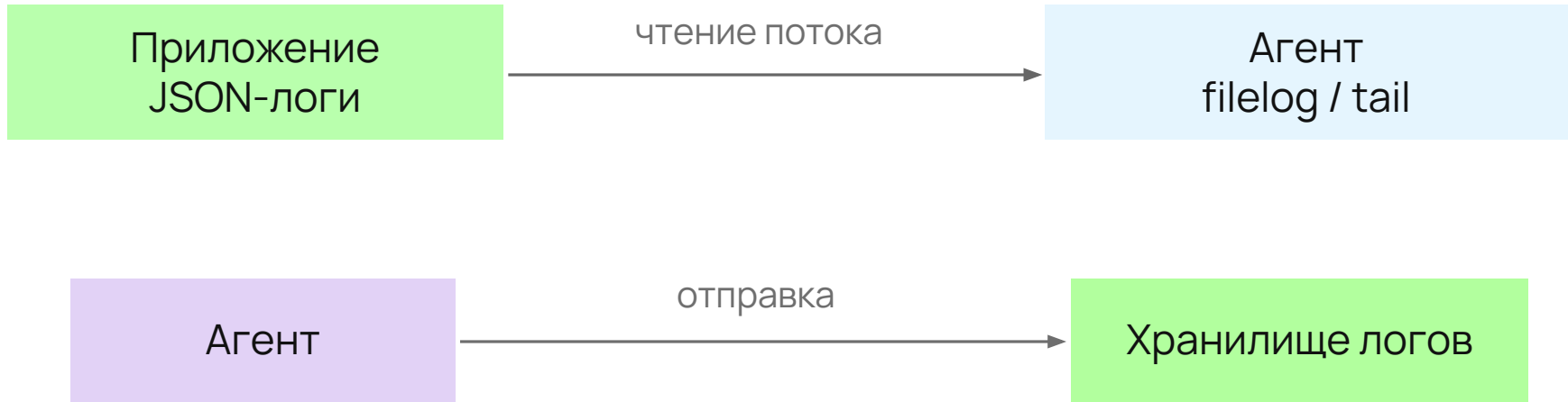


Pull: scrape



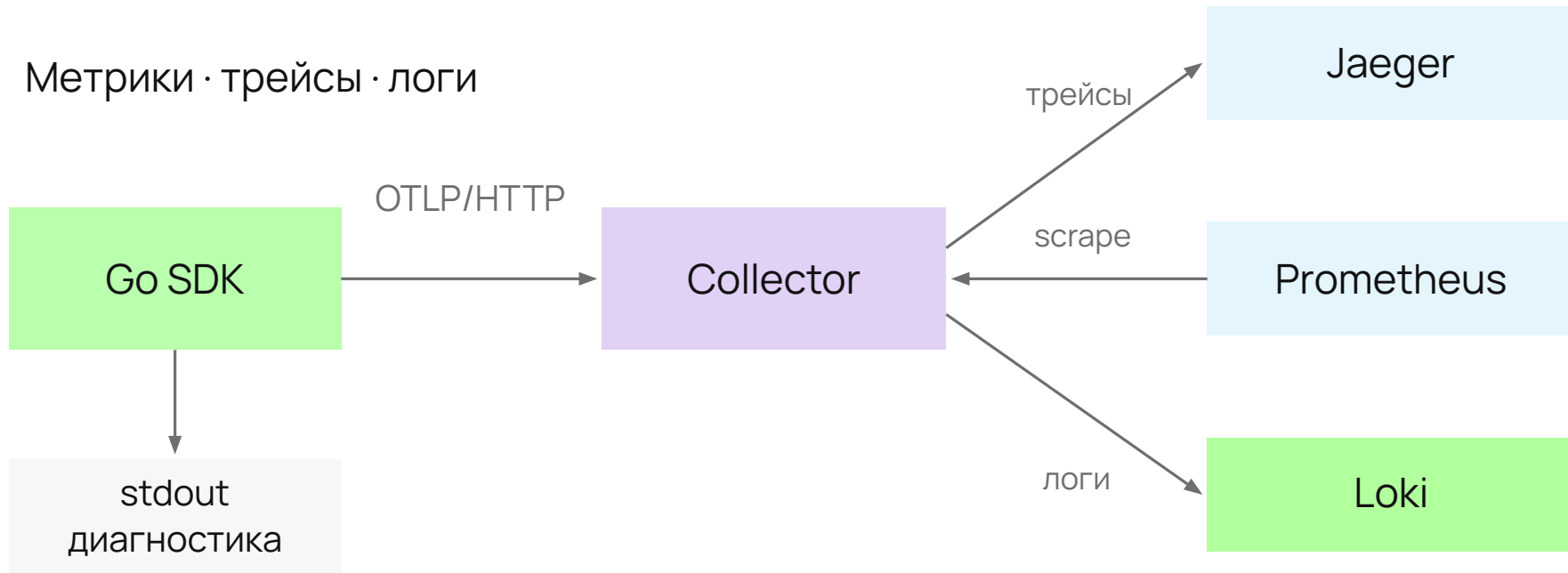
Логи: альтернативный путь через агент

Сбор stdout / файлов



Путь телеметрии на стенде

Метрики · трейсы · логи



Grafana: Prometheus + Jaeger + Loki

От сигнала к действию

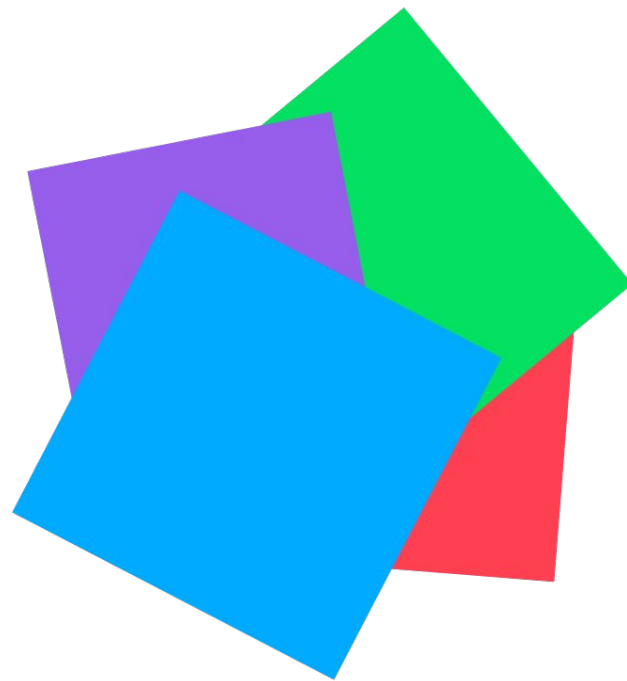
Метрика → отклонение → алерт

Трейс + лог → исследование причины

SLI → SLO → бюджет ошибок

Итоги:

- Разобрались как строится мониторинг приложения
- Узнали об использовании мониторинга для определения и реагирования на проблемы с системой



Материалы для самостоятельного изучения

[Google SRE Workbook · SLI / SLO](#)

[OTel · ExponentialHistogram](#)

[Prometheus · rate / increase / histograms](#)

[VictoriaMetrics · MetricsQL](#)